Entity Resolution Platform

ER-PLATFORM-001 — architecture, every concept used, and why each choice was made.

Multi-tenant record linkage for Indian school data. FastAPI + PostgreSQL 16/pgvector + Redis + Celery + XGBoost + sentence-transformers + Splink, with a Next.js 15 frontend. Written for an engineer joining this codebase.

1. What the system does

A customer uploads a CSV of school records. The same real-world school appears several times, spelled differently each time — `Govt. Higher Primary School Unit 7` and `Government Higher Primary School Unit 7` are one school, and nothing in the data says so. The platform decides which rows are the same place, and produces a mapping from every "dirty" record id to a surviving "master" id.

This is **entity resolution** (also called record linkage or deduplication). The hard part is not comparing two records — it is that comparing every row against every other row is quadratic. A 50,000-row file is 1.25 billion pairs. The whole architecture is shaped around avoiding that number while still finding the matches.

> CORE IDEA **Cheap filters first, expensive judgement last.** Each stage removes work from the next. Classification throws away junk before it is embedded; blocking narrows 1.25 billion pairs to a few thousand before they are scored; the confidence gate decides which of those are worth a paid API call or a human's attention. Every stage exists to protect the one after it.

2. The pipeline, end to end

```
CSV upload (HTTP 202, returns immediately)
  │
  ├─▶ 1. INGEST — parse CSV in chunks, store each row verbatim as JSONB
  ├─▶ 2. CLASSIFY — XGBoost: Valid / Junk / Missing_Context
  │     (junk never reaches the expensive stages)
  ├─▶ 3. EMBED — normalise text, then 2 × 768-dim vectors per record
  ├─▶ 4. BLOCK — pgvector HNSW nearest-neighbour search → candidate pairs
  │     → connected components → clusters
  ├─▶ 5. MATCH — Splink scores every pair inside every cluster → confidence
  └─▶ 6. GATE — route each cluster by its tenant's thresholds:
       ≥ 0.950 → AUTO_MERGED (no external call)
       ≥ 0.750 → PENDING_API (validate against Google Places)
       < 0.750 → PENDING_REVIEW (a human decides)
```

Each stage is a separate Celery task on its own queue with its own worker pool, so the GPU-ish embedding work and the rate-limited API work scale independently.

3. Multi-tenancy and Row-Level Security

Many customers share one database. The requirement (NFR-02) is that *a single forgotten* `WHERE tenant_id = ?` *must not be a data breach.* That is enforced in two independent layers.

Layer 1 — application scoping

The tenant id comes only from a verified JWT claim, never from a request body or query parameter. Every request opens a session bound to that tenant.

Layer 2 — PostgreSQL Row-Level Security (RLS)

RLS is a Postgres feature where the database itself attaches an invisible `WHERE` clause to every query on a table. The policy reads a per-transaction session variable:

```
CREATE POLICY tenant_isolation ON school_records
  USING      (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

CONCEPT	WHAT IT MEANS HERE
`USING`	Governs what rows a query can *see* (SELECT, and the read half of UPDATE/DELETE).
`WITH CHECK`	Governs what rows a query can *write*. Without it a tenant could insert rows stamped with someone else's id.
`FORCE ROW LEVEL SECURITY`	Applies the policy to the table *owner* too. Plain `ENABLE` silently exempts the owner — policies look correct and leak anyway.
Three DB roles	`er_owner` (migrations only), `er_app` (runtime, `NOBYPASSRLS`), `er_platform` (`BYPASSRLS`, control plane only).

TRAP The policy is written as `current_setting('app.tenant_id')::uuid` with no `missing_ok` flag, so a query with no tenant bound **raises**. The safer-looking `NULLIF(current_setting(..., true), '')` returns NULL, matches nothing, and returns *zero rows silently* — which reads as "this tenant has no data" instead of "you forgot to bind a tenant".

BUG THIS CAUGHT Five of six write sites had forgotten to set `tenant_id` at all. The `WITH CHECK` clause rejected the very first insert, which is exactly what the second layer is for. Found only by running against a real container — no mock would have.

4. The data model

TABLE	PURPOSE	NOTABLE
`tenants`	Root of the tenancy tree	Per-tenant confidence thresholds and monthly spend cap
`jobs`	One uploaded CSV	`status` + `stage_progress` JSONB counters
`school_records`	One CSV row	`raw_row` JSONB written once, never mutated
`record_embeddings`	Two 768-dim vectors	`vector(768)` columns, HNSW-indexed
`clusters` / `cluster_members`	Candidate groups	`confidence_score NUMERIC(4,3)`
`google_places_validations`	API response cache	Keyed `(tenant_id, query_hash)`
`human_feedback`	Steward verdicts	Training data, not an audit log
`reference_pincodes`	All-India PIN lookup	**Global — no tenant, no RLS**

> **WHY** **`raw_row` is immutable.** Everything else on a record is derived from it, so re-running a stage re-derives rather than re-reads, and the customer's original upload is always recoverable. Derived columns (`name_normalized`, `classification`) are nullable because they are populated by a later stage — defaulting them to `0` or `''` would be a lie the UI renders as fact.

ALEMBIC MIGRATIONS

Schema changes are versioned Python files applied in order (`0001` → `0004`). They run as `er_owner`; the runtime role deliberately has no `CREATE` privilege. Migration `0002` installs RLS and ends with a guard that **fails the migration** if any table lacks a policy — so a table added later cannot ship unprotected.

5. Stage 2 — The classifier (weak supervision)

A gradient-boosted tree (XGBoost) sorts every row into Valid, Junk or Missing_Context from five cheap features: name length, token count, count of missing required fields, whether the pincode is a placeholder, and whether the name matches a sentinel pattern (`N/A`, `TEST`, `XXXX`).

The cold-start problem

A new customer has no labelled data, so there is nothing to train on. The solution is **weak supervision** (the Snorkel pattern): write many imperfect rules, let them vote, and train on the consensus.

```
lf_placeholder_name      → JUNK            (weight 2.0)
lf_empty_name            → JUNK            (weight 2.0)
lf_name_has_no_letters   → JUNK            (weight 1.5)
lf_unusable_pincode      → MISSING_CONTEXT (weight 1.5)
lf_several_missing_fields→ MISSING_CONTEXT (weight 2.0)
lf_complete_record       → VALID           (weight 2.0)
lf_descriptive_name      → VALID           (weight 1.0)
```

Each labelling function votes for one class or *abstains*. Votes are combined by weighted majority. Rows where the rules disagree, or all abstain, are **dropped from training** rather than guessed at — those are exactly the rows a human should see. Labels produced this way are stamped `label_source = 'PSEUDO'` so a later retrain can discard them without contaminating real ground truth.

> **BUG FOUND IN TESTING** A row that was empty in *every* field was labelled `Missing_Context`, because the missing-field rules outvoted the junk rules on sheer count of absent fields. A row with nothing in it is not "a real school described thinly" — it is junk. Fixed by gating the Missing_Context rules on the record having a usable name.

6. Stage 3 — Embeddings and normalisation

An **embedding** turns text into a vector of numbers positioned so that similar meanings land near each other. The model is `paraphrase-multilingual-mpnet-base-v2`, chosen because Indian school names arrive transliterated inconsistently and sometimes in Devanagari.

Two independent vectors are stored per record — one for name, one for address — so that a generic name in two different cities is never treated as one candidate.

Normalisation runs first

```
"Govt. Sr. Sec. School, Hosur"  →  "government senior secondary school hosur"
"KV No 2 Jalahalli"             →  "kendriya school number 2 jalahalli"
"Kendriya Vidyalaya No. 2 …"    →  "kendriya school number 2 jalahalli"   ← same string
```

Lowercase, strip punctuation, expand abbreviations from a table of ~45 Indian school terms, applied repeatedly until the string stops changing. That *fixed-point* detail matters: `kv → kendriya vidyalaya → kendriya school` takes two passes, and a single pass would leave the two spellings as different strings.

> **BUG FOUND IN TESTING** The first version stripped punctuation with a "keep word characters" rule. Python's `\w` does not count Devanagari vowel marks as word characters, so `केंद्रीय विद्यालय` came back as a row of bare consonants — silently destroying exactly the multilingual text the model was chosen for. Fixed by naming the punctuation to remove instead of naming the characters to keep.

7. Stage 4 — Blocking (the quadratic problem)

Blocking is the step that makes entity resolution tractable: instead of comparing all pairs, find each record's nearest neighbours in vector space and only compare those.

CONCEPT	MEANING
pgvector	A Postgres extension adding a `vector` column type and distance operators.
Cosine distance (`<=>`)	`1 - cosine similarity`. 0 = identical direction, 2 = opposite.
HNSW	Hierarchical Navigable Small World — a graph index for *approximate* nearest-neighbour search. Trades exactness for speed.
ANN	Approximate Nearest Neighbour. "Approximate" is the important word.
Connected components	If A–B and B–C are pairs, then A, B, C form one cluster even though A–C was never compared.

THE BIG TRAP **pgvector applies the HNSW approximation *before* the RLS tenant filter.** Ask for the 10 nearest neighbours and the index returns 10 rows from the *whole* table — which are then filtered down to the ones your tenant may see. If another tenant's rows occupy the nearest slots, you get far fewer than 10, or zero, **with no error and no warning anywhere**.

Mitigated two ways: over-fetch (`LIMIT top_k × oversample`, truncate in Python) and raise `hnsw.ef_search`, which controls how much of the graph is explored. Both are verified by an integration test that puts 300 of another tenant's records nearer to the probe than any of its own.

8. Stage 5 — Probabilistic matching (Splink)

Vector distance says two records are *worth comparing*. It does not say they are the same school — two generic government school names are close in vector space whether or not they are the same institution. The actual score comes from Splink, which implements the **Fellegi–Sunter** model.

How Fellegi–Sunter works

TERM	MEANING
m probability	P(fields agree | the pair *is* a match). Learned by Expectation-Maximisation.
u probability	P(fields agree | the pair is *not* a match). Learned by sampling random pairs.
λ (lambda)	P(two randomly drawn records match at all) — the prior everything else updates.
Match weight	`log₂(m/u)` summed across fields. Agreement on a *rare* value is worth far more than on a common one.

The comparison set is name (Jaro-Winkler bands), address (looser thresholds), and pincode — where a custom comparison exploits the hierarchy of an Indian PIN: exact match, then same sorting district (first 3 digits), then same postal region (first digit).

> **BUG FOUND IN TESTING** Skipping the λ-estimation step left genuine duplicates scoring **0.043** while non-duplicates scored 0.000. The ordering was right and the numbers were useless — and since auto-merge triggers at 0.950, *nothing would ever have merged*. Adding the step moved true duplicates to **0.87–0.91**.

> **WHY** **A cluster's score is the *minimum* pairwise score inside it, not the mean.** A cluster is a transitive closure — A and C may never have been compared — so it is only as trustworthy as its weakest join. A mean would let one strong pair carry a cluster containing a pair the model scored near zero.

9. Stage 6 — The confidence gate and cost control

Google Places charges per call. Three independent controls sit in front of every request:

1. **The confidence gate** decides *which* clusters may ask at all — only the middle band.
2. **The cache** (`google_places_validations`) decides whether the question was already paid for. Negative results are cached too: a school Google has never heard of is still an answer, and re-buying it every run is the most expensive possible reading of a cache.
3. **The field mask** restricts the request to `name,formatted_address,geometry` . Google bills by field group, so the mask is a line item — and fields not requested are also fields never stored.

Token bucket rate limiting

A **token bucket** holds a number of tokens that refill at a fixed rate. Each API call takes one; empty bucket means wait. Implemented as an atomic Redis Lua script, because with two workers and a retry storm a check-

then-decrement race is not theoretical — and every leaked token is a billable call.

Buckets are **per tenant**: with one shared bucket, a single customer's 50k-row backlog would drain the tokens and stall everyone else's validation.

10. Orchestration — Celery, queues, idempotency

Celery is a distributed task queue. The API does not do slow work; it writes a row, pushes a message to **Redis** (the broker), and returns HTTP 202. Worker processes consume messages and do the work.

```
ingest · classify · embeddings · blocking · matching
google_places · retrain · export · orchestration
```

Nine queues, each with its own pool and retry policy. `google_places` is capped at concurrency 2 — that cap is a *spend* control, not a throughput setting.

Idempotency

Celery is configured with `task_acks_late`: a worker that dies mid-task has its message redelivered. Re-running a stage is therefore *normal*, not exceptional, and every stage must converge rather than duplicate:

- **Ingest** — detects existing records and re-classifies in place instead of re-inserting.
- **Embeddings** — `INSERT … ON CONFLICT DO UPDATE` keyed on `record_id`.
- **Blocking** — deletes and rebuilds, but protects clusters a human already decided.
- **Matching** — overwrites scores in place.

Stages hand off **through Postgres, not through message payloads**. A 50k-row job produces ~1 million candidate pairs; routing those through Redis would turn the broker into a data channel it was never sized for.

> **BUG FOUND IN TESTING** A self-inflicted deadlock. When a stage failed, the error handler opened a *second* database connection to mark the job FAILED — while the failing transaction still held a lock on that same row. Postgres could not break it (one side was waiting on the client, not on a lock, so there was no cycle to detect) and the worker hung for over ten minutes. Fixed by rolling back the failed transaction *before* opening the new one.

11. The API layer

ENDPOINT	PURPOSE
`POST /v1/jobs`	Upload CSV, create job, enqueue. Returns **202**.
`GET /v1/jobs/{id}`	Status and per-stage progress.
`GET /v1/jobs/{id}/stats`	Aggregates, computed in SQL.
`GET /v1/jobs/{id}/clusters`	Review queue, keyset-paginated.
`GET /v1/jobs/{id}/export`	Streaming CSV of dirty id → master id.
`GET /v1/clusters/{id}`	Cluster with all member records.
`POST /v1/clusters/{id}/feedback`	MERGE / REJECT / SPLIT verdict.

Keyset pagination

The review queue pages by **cursor**, not `OFFSET`. The reason is specific to this screen: stewards resolve clusters *while* paging through them, so rows leave the filtered set underneath the cursor. Under `OFFSET`, every resolution shifts the remaining rows left and page 2 silently skips whatever moved into page 1's window — those clusters are never seen again, and nothing reports a problem.

The cursor encodes `(created_at, id)`. The trailing id makes the sort *total*, so two clusters created in the same microsecond cannot cause a page to repeat or skip.

Streaming export

`StreamingResponse` over a server-side cursor, so memory stays at one fetch window rather than the whole job. Records in no cluster are exported as `UNRESOLVED_SINGLETON` — an export that silently omitted them would not reconcile against the customer's input file.

Authentication

A **JWT** (JSON Web Token) is a signed, tamper-evident bundle of claims. This one carries `sub`, `tenant_id`, `role`, `exp`, `aud`, `iss`. It is verified on every request, and the tenant id it carries is the *only* source of tenancy anywhere in the request path.

Auth failures return an identical 401 for "expired" and "bad signature" — the difference is useful to an attacker and not to a legitimate caller.

12. The frontend

CONCEPT	WHAT IT DOES HERE
Next.js App Router	File-system routing; folders under `app/` become URLs.
Server vs Client Components	Server components render on the server; `"use client"` marks components needing state or effects.
Tailwind v4	Utility CSS classes rather than separate stylesheets.
Rewrites (proxy)	`/v1/*` is proxied to FastAPI so the browser sees one origin.

WHY THE PROXY The backend installs no CORS middleware, so a browser at `:3000` cannot call `:8000` directly — the request is blocked before FastAPI sees it. Proxying through Next makes every call same-origin, and leaves CORS as a production decision (an allow-list of real origins) rather than something loosened for convenience.

WHY XMLHTTPREQUEST FOR UPLOAD `fetch` cannot report upload progress. For a file that may be hundreds of megabytes, a UI showing nothing between "clicked" and "done" looks broken — and the user's instinct is to click again, which uploads it twice.

The review screens

The queue table pages by the backend's opaque cursor. The resolution screen shows member records side by side with disagreements highlighted *relative to the proposed master*, because the question being answered is "should these collapse into *that* one".

TRAP `PENDING_REVIEW` is a *cluster* status; `AWAITING_REVIEW` is a *job* status. They describe the same moment from opposite sides, and filtering clusters by `AWAITING_REVIEW` returns an empty page rather than an error.

Also: scores arrive as JSON **strings** (`"0.842"`) because they are SQL `NUMERIC` . Sorting them unparsed gives lexicographic order, where `"0.9" > "0.85"` is wrong.

13. Testing strategy

LAYER	APPROACH
Unit (ML, helpers)	No database. Pure logic, deterministic fixtures.
Blocker	Mocked session, but statements compiled against the real PostgreSQL dialect so invalid SQL fails locally.
Matcher	Real Splink on in-process DuckDB — the production backend, not a stand-in.
API	Real FastAPI test client; auth boundary walked from the live OpenAPI schema.
Integration	Real Postgres, Redis, Celery, XGBoost, Splink in Docker.

242 tests. The integration suite is the one that earned its cost: it found the missing `tenant_id` writes, the self-deadlock, and the fact that an unseeded `reference_pincodes` table silently classifies every upload as `Missing_Context` — producing a job that succeeds having resolved nothing.

> **PRINCIPLE** Tests assert *properties*, not snapshots — "duplicates outrank non-duplicates", not "this pair scores 0.847". A snapshot test fails on every dependency bump without anything being wrong.

14. Glossary

TERM	DEFINITION
Entity resolution	Deciding which records refer to the same real-world thing.
Blocking	Narrowing the candidate pair set so scoring is not quadratic.
Embedding	Text represented as a vector positioned by meaning.
HNSW	Graph index for fast approximate nearest-neighbour search.
Cosine distance	`1 - cosine similarity`; the `<=>` operator.
RLS	Row-Level Security — per-row access policies enforced by Postgres.
GUC	Grand Unified Configuration — a Postgres session variable, here `app.tenant_id`.
Weak supervision	Training on noisy rules instead of hand-labelled data.
Labelling function	One imperfect rule that votes for a class or abstains.
Fellegi–Sunter	The probabilistic record-linkage model Splink implements.
m / u probability	Agreement rate among matches / among non-matches.
EM	Expectation-Maximisation — fits parameters without labels.
Idempotency	Running twice leaves the same state as running once.
Keyset pagination	Paging by a position in the data, not a row offset.
Token bucket	Rate limiter: tokens refill at a rate, each action spends one.
Dead letter	Where a task goes after exhausting its retries, so it can be diagnosed.
JWT	Signed token carrying identity claims.
202 Accepted	"Work queued", not "work finished".